

## CSCE 753 COMPUTER VISION · MODULE 07 · LECTURE NOTES

# Motion and Tracking

*Correspondence over time.*

## MODULE OUTCOMES

- Derive the optical flow constraint and the aperture problem.
- Explain Lucas–Kanade and its conditioning.
- Explain coarse-to-fine estimation.
- Explain filtering for tracking, and what a Kalman filter assumes.
- Distinguish tracking from detection, and explain when each fails.

## 1 Optical flow

$$I(x, y, t) = I(x + u, y + v, t + 1) \quad (\text{brightness constancy})$$

$$I_x u + I_y v + I_t = 0 \quad (\text{first order})$$

One equation, two unknowns, at every pixel. Flow is under-determined pointwise — **which is Module 03 §1's aperture problem, now as an algebraic fact rather than an intuition about windows. Only the flow component along the image gradient is recoverable** (the normal flow); the component perpendicular to the gradient does not appear in the equation at all. **This is exactly why a rotating barber pole appears to move upward:** the stripes' gradients are perpendicular to the stripes, so the recoverable motion is across them, and the true horizontal rotation is invisible. **The illusion is not a quirk of human vision; it is the correct answer to an under-determined problem,** and any algorithm using only local information makes the same report.

**LUCAS-KANADE: ASSUME THE FLOW IS CONSTANT OVER A WINDOW**

**One equation per pixel is insufficient, so assume that all pixels within a small window share a single flow vector** and solve the resulting overdetermined system by least squares. The assumption is the prior that resolves the under-determination (Module 01 §3).

**The normal equations involve the structure tensor again** — exactly the  $2 \times 2$  gradient covariance matrix of Module 03 §1's Harris derivation, summed over the same kind of window.

**So flow is recoverable precisely where Harris finds corners**, and unrecoverable on edges (one eigenvalue near zero) and flat regions (both near zero). **'Good feature to track' and 'good feature to match' are literally the same property**, and **Shi-Tomasi's refinement of Harris was derived from exactly this observation** — use the smaller eigenvalue directly, since that is what bounds the tracking accuracy.

**And the matrix's condition number is the uncertainty of the estimate. Report it, and discard tracks whose conditioning is poor**, rather than propagating a confident wrong velocity into Module 05's bundle adjustment — which is Module 01 §2's stability problem, with a cheap and local diagnostic available.

## 2 Large motion

THE PROBLEM: the flow equation came from a FIRST-ORDER Taylor expansion, valid only for displacements of about one pixel. A 30-pixel motion violates it completely, and the resulting estimate is not merely inaccurate -- it is unrelated to the true motion.

THE FIX: build an image pyramid. At  $1/32$  resolution, a 30-pixel motion IS about one pixel.

```

estimate flow at the coarsest level
upsample the flow field, scale it by 2
WARP the next image by the current estimate
estimate the RESIDUAL flow at this level
repeat down to full resolution

```

Each level only ever estimates a small residual, so the linearisation is valid everywhere it is used.

THIS IS NOT A SPEED OPTIMISATION. Without it the method does not work at all for realistic motion. It is also precisely CSCE 669 Module 07's trust-region idea: take a step only as large as the local model justifies, then rebuild the model.

AND IT FAILS when a SMALL object moves FAR. The object disappears at coarse scales, so the coarse estimate does not see it, and the fine levels are initialised from an estimate that is already in the wrong basin.

**The small-fast-object failure is intrinsic to coarse-to-fine estimation** rather than a tuning problem, and **it is a large part of why learned flow methods won on benchmarks that contain such motion** — a learned matcher with a large receptive field or an all-pairs correlation volume is not constrained to search locally, so it can find a correspondence that no coarse-to-fine scheme could reach. **Which is the same pattern as Module 03 §4: learning won where the classical method had a structural, identifiable weakness**, and that is where to expect it to win in general.

### 3 Tracking over time

#### A KALMAN FILTER IS OPTIMAL UNDER ASSUMPTIONS YOU SHOULD CHECK

**Predict the state forward using a motion model, then correct it with the new measurement, weighting prediction and measurement by their respective covariances.** That is the entire algorithm, and it is two matrix equations.

**It is provably optimal when the dynamics are linear, the noise is Gaussian, and the motion model is correct. All three assumptions fail in real tracking** — objects accelerate unpredictably, outliers are not Gaussian, and a constant-velocity model is wrong whenever anything interesting happens.

**Non-linear dynamics are handled by the extended or unscented variants; genuinely multi-modal uncertainty requires a particle filter**, because a **Gaussian cannot represent 'the object is in one of two places'** — it represents that as 'the object is probably between them', which is the one place it is not.

**And the failure mode is overconfidence. A filter whose model is wrong reports a small covariance while being badly wrong**, because the covariance reflects the model's own self-assessment and not reality — and once confident, it *rejects the correct measurement as an outlier* and diverges further. **So monitor the innovation (the measurement residual), not the reported covariance**: persistently large innovations mean the model is wrong, whatever the covariance claims. This is the same lesson as CSCE 633 Module 12's calibration — a confident estimator is not a correct one.

|          | Tracking                                                                                                         | Detection every frame                                                                                  |
|----------|------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Uses     | The previous position, so the search region is small.                                                            | Nothing from the past; searches the entire frame.                                                      |
| Cost     | Cheap — a local search.                                                                                          | Expensive — a full inference pass.                                                                     |
| Identity | Maintained naturally — the track is the identity.                                                                | <b>Must be re-associated across frames</b> , which is its own assignment problem (CSCE 669 Module 11). |
| Fails by | <b>DRIFT</b> — the window slides gradually onto the background and the track is lost without any discrete event. | Missed detections and flicker, which are visible and discrete.                                         |
| Recovery | <b>Cannot recover once lost</b> , because its only reference was the thing it lost.                              | <b>Recovers automatically on the next frame</b> , with no memory of the failure.                       |

**So real systems do both:** track for cheapness and for identity continuity, and detect periodically to correct drift and to recover after occlusion. **Tracking-by-detection is the standard architecture**, and the interesting engineering is in the association step between the two.

### 4 Motion in real systems

- **Rolling shutter.** Most CMOS cameras expose image rows at different times, spread over milliseconds, so a fast-moving object is sheared and a fast-rotating camera produces a warped frame. **The rigid**

**single-pose assumption behind Modules 04 and 05 is then false**, and the symptom is a reconstruction that will not converge for no visible reason. **Model it, or use a global-shutter camera.**

- **Motion blur.** The feature being tracked is smeared along the direction of motion, **which biases its localisation along that same direction** — a systematic error correlated with the quantity being estimated, which is the worst kind.
- **Illumination change.** Brightness constancy fails under auto-exposure adjustment, mains flicker, or a cloud passing. **Lock the exposure if the application permits**; otherwise use a cost that is invariant to monotonic intensity change (Module 06 §1's census).
- **Occlusion.** **The track must be suspended, not continued. Continuing to update through an occlusion is exactly how trackers end up following the occluder** instead of the target, and then confidently reporting it.
- **And measure drift directly.** **Track a point out and back to its starting frame and report the closure error in pixels. This needs no ground truth, takes minutes, and is the honest measure of a tracker** — which is why it is worth building into the development loop rather than reserving for evaluation.

#### WHERE MOTION CONNECTS TO THE REST OF THE COURSE

**Flow gives dense correspondence, which is dense stereo's input when the two frames come from a moving camera — structure from motion and optical flow are the same problem under different parameterisations**, one solving for a sparse set of poses and points and the other for a dense displacement field.

**Tracked features supply the tracks of Module 05 §1, so a video is a structure-from-motion problem with free and unusually reliable correspondence** — temporal adjacency makes matching far easier than it is between unordered photographs.

**And PnP against a map at 60 Hz is SLAM** (Module 05 §1), which is what that module was building toward and what every phone AR system runs continuously — the direct bridge from this course into CSCE 650's tracking requirements.

**So Modules 03 through 07 are one subject. Correspondence — across space or across time — and everything geometric in this course depends entirely on it. Which is why correspondence failure, not algorithm failure, is the cause of most broken geometric pipelines**, and why the diagnostic habit is always to inspect the matches first.

## Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

| Resource                                         | What it covers for this module                                                                |
|--------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Szeliski — Computer Vision, chapter 9 (free PDF) | <b>Motion estimation, optical flow, and the pyramid scheme</b> — the reference for §1 and §2. |

| Resource                                                        | What it covers for this module                                                                                                 |
|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Baker & Matthews — Lucas-Kanade 20 Years On (free)              | <b>The §1 method, developed properly</b> , including the inverse-compositional formulation that makes it fast.                 |
| Welch & Bishop — An Introduction to the Kalman Filter (free)    | <b>The clearest free introduction to §3</b> , with the assumptions stated explicitly.                                          |
| Teed & Deng — RAFT: Recurrent All-Pairs Field Transforms (free) | <b>The learned flow method that addresses §2's intrinsic failure</b> , by correlating all pairs rather than searching locally. |

## Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Derive the flow constraint** and show that only the gradient-aligned component is determined.
- 2 **Render a rotating barber pole** and compute its optical flow. Confirm it points upward.
- 3 **Implement Lucas–Kanade** and visualise the structure tensor's smaller eigenvalue as a trackability map.
- 4 **Compare it against your Harris response** from Module 03 and confirm they agree.
- 5 **Estimate flow for a 30-pixel motion without a pyramid** and report the result.
- 6 **Add the pyramid** and report the improvement.
- 7 **Construct a small fast-moving object** and show the pyramid fails on it.
- 8 **Implement a constant-velocity Kalman filter** and track an object that accelerates. Plot the innovation.
- 9 **Show the filter rejects correct measurements** once its model diverges.
- 10 **Run the out-and-back drift test** on your tracker and report the closure error.

## Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Derive the brightness constancy constraint and state the aperture problem algebraically.
- 2 Why does a barber pole appear to move upward?
- 3 What does Lucas–Kanade assume, and what matrix appears in its solution?
- 4 Why are good features to track the same as good features to match?
- 5 Why is coarse-to-fine not a speed optimisation?
- 6 What motion does coarse-to-fine intrinsically fail on?
- 7 State the Kalman filter's three assumptions and its failure mode.

- 8 Why monitor the innovation rather than the covariance?
- 9 Compare tracking against per-frame detection on five axes.